

자료구조 실습 보고서

[제 05 주] 별의 집합

제출일 : 2018 년 4 월 9 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘, 자료구조

Set 자료 구조를 배열, 연결 체인 두 가지 방법을 이용하여 구현하였다.

Set 은 중복을 허용하지 않는 구조로, 원소들 사이의 순서는 고려하지 않는다.

이는 Bag 에서 중복만 제거한 것과 같다.

- `public boolean add(E anElement)`

집합이 꽉 찼거나, 추가하고자 하는 원소를 이미 가지고 있다면 `false` 를 반환한다.

아니라면 비어있는 공간의 가장 앞쪽에 원소를 추가하거나 (ArraySet),

연결체인의 가장 앞쪽인 head 에 원소를 추가한다. (LinkedSet)

Bag 의 add 함수에 중복 여부를 확인하는 부분만 추가하였다.

- `public boolean equals(Object aStar)`

최상위 Object 클래스의 메소드를 오버라이딩 한 것이기 때문에 매개변수로 Object 형을 받아야 하며, 함수 내부에서 이를 사용할 때는 형변환을 해준다.

비교하고자 하는 원소의 자료형이 Star 가 아닐 경우엔 `false` 를 반환한다.

Star 형이라면, 좌표와 이름중 같은 것이 있을 때 `true` 를 반환하며 둘 다 다르다면 `false` 를 반환한다.

b. 함수 설명서

- StarCollection_ArraySet

- AppController

- `public void run()` : 프로그램 실행

- AppView

- `public int inputInt()` : 숫자 하나 입력받아 반환

- `public String inputString()` : 문자열 입력받아 반환

- `public void outputMessage(String aMessage)` : 문자열 받아 출력

- `public void outputstar(String aStarName, int aX, int aY)` : Star 이름, 좌표 받아 출력

- `public void outputStarExistence(String aStarName)` : Star 이름 받아 존재 여부 출력

- `public void outputStarExistence(int aX, int aY)` : Star 좌표 받아 존재 여부 출력

- `public void outputNumberOfStars(int aStarCollectorSize)` : 저장된 Star 개수 받아 출력

- ArraySet

- `public ArraySet()` : 최대 크기를 기본값인 100 으로 설정한 후 배열 집합 생성

- `public ArraySet(int givenCapacity)` : 최대 크기를 직접 설정한 후 배열 집합 생성
- `public int size()` : 집합 크기 반환 (Getter)
- `public boolean isEmpty()` : 집합 비어있는지 여부 반환
- `public boolean isFull()` : 집합이 다 찼는지 여부 반환
- `public E elementAt(int anOrder)` : 인자로 받은 인덱스에 있는 원소 반환 (Getter)
- `public boolean doesContain(E anElement)` : 인자로 받은 원소와 같은 값의 원소가 있는지 여부 반환
- `public boolean add(E element_p)` : 인자로 받은 원소를 집합에 추가
- `public boolean remove(E element_p)` : 인자로 받은 원소와 같은 값을 갖는 원소 1 개 제거
- `public E removeAny()` : 맨 뒤의 원소 하나 삭제하고 삭제한 원소 반환
- `public void clear()` : 집합 비움

■ Star

- `public Star(int givenX, int givenY)` : 좌표 받아 설정 (생성자)
- `public Star(String givenStarName)` : 이름 받아 설정 (생성자)
- `public Star(int givenX, int givenY, String givenStarName)` : 좌표와 이름 받아 설정(생성자)
- `public int xCoordinate()` : x 좌표 반환 (Getter)
- `public int yCoordinate()` : y 좌표 반환 (Getter)
- `public String starName()` : 이름 반환 (Getter)
- `public void setXCoordinate(int newX)` : x 좌표 설정 (Setter)
- `public void setYCoordinate(int newY)` : y 좌표 설정 (Setter)
- `public void setStarName(String newStarName)` : 이름 설정 (Setter)
- `public boolean equals(Object aStar)` : 인자로 받은 값이 Star 형이 아니라면 false 반환
Star 형이 맞다면, 좌표, 이름 비교한 뒤에 하나라도 같으면 true 반환, 전부 다르면 false 반환

● StarCollection_LinkedSet

: ArraySet 대신에 LinkedSet 과 node 가 들어가고, 나머지 메소드 전부 같음

■ LinkedSet

- `public LinkedSet()` : 집합의 현재 크기와, 맨 앞의 노드 초기화 (생성자)
- `public int size()` : 집합 크기 반환 (Getter)
- `public boolean isEmpty()` : 집합 비어있는지 여부 반환
- `public boolean isFull()` : false 반환
- `public Element elementAt(int givenOrder)` : 인자로 받은 인덱스에 있는 원소 반환 (Getter)
- `public boolean doesContain(Element givenElement)` : 인자로 받은 원소와 같은 값의 원소가 있는지 여부 반환
- `public boolean add(Element givenElement)` : 인자로 받은 원소를 집합에 추가

- `public boolean remove(E element_p)` : 인자로 받은 원소와 같은 값을 갖는 원소 1 개 제거
- `public Element removeAny()` : 맨 앞의 원소 삭제한 후, 삭제한 원소 반환
- `public void clear()` : 집합 초기화

■ Node

- `public Node()` : 원소와 다음 노드 위치 정보 초기화 (생성자)
- `public Node(Element givenElement)` : 원소 직접 설정, 다음 노드 위치 정보 초기화 (생성자)
- `public Node(Element givenElement, Node<Element> givenNext)` : 원소와 다음 노드 위치 정보 직접 설정 (생성자)
- `public Element element()` : 현재 노드의 원소 반환 (Getter)
- `public Node<Element> next()` : 다음 노드 위치 정보 반환 (Getter)
- `public void setElement(Element givenElement)` : 현재 노드의 원소 설정 (Setter)
- `public void setNext(Node<Element> givenNext)` : 다음 노드 설정 (Setter)

c. 종합 설명서

이번 과제는 Set 자료 구조를 배열 또는 연결체인을 이용해서 구현하는 것이다.

Set 구조는 기본적으로 Bag 에서 중복을 제거한 것과 같기 때문에, `add()`를 중복을 허용하지 않도록 수정하였다.

프로그램이 시작되면, 메뉴를 입력받으며 그에 따라 별 추가, 이름 입력하여 삭제, 무작위 삭제, 현재 개수 출력, 이름으로 존재 여부 탐색, 좌표로 존재 여부 탐색, 종료를 할 수 있다.

두 프로그램은 단지 Set 의 구현 방법만 바꾼 것이고 다른 모듈들과 상호작용하는 방법은 동일하기 때문에, 연결 체인을 이용하여 만들 때에는 `ArraySet` 을 대체하는 것 외에는 다른 부분은 수정하지 않았다.

2. 프로그램 장단점 분석

장점 : 지난 번에는 정수를 입력받을 때 입력값의 형식이 맞지 않으면 `Exception` 이 발생하며 프로그램이 비정상 종료되었지만, 이번에는 `try ~ catch` 문을 사용하였기 때문에 오류가 발생 시 입력을 다시 받는다.

단점 : 한글을 입력받을 시 입력이 잘 되지 않는다.

3. 실행 결과 분석

: 두 프로그램은 자료구조만 바꾼 것이므로 실행 결과는 같다.

a. 입력과 출력

<추가>

```
< 별의 집합을 시작합니다. >

1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 1

-   [별 추가]   -
- x 좌표를 입력하시오 : 1
- y 좌표를 입력하시오 : 1
- 별의 이름을 입력하시오 : a

1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 1

-   [별 추가]   -
- x 좌표를 입력하시오 : 2
- y 좌표를 입력하시오 : 2
- 별의 이름을 입력하시오 : b

1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 1

-   [별 추가]   -
- x 좌표를 입력하시오 : 3
- y 좌표를 입력하시오 : 3
- 별의 이름을 입력하시오 : c
```

<오류>

```
1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 1

-   [별 추가]   -
- x 좌표를 입력하시오 : 1
- y 좌표를 입력하시오 : 1
- 별의 이름을 입력하시오 : x
ERROR : 추가 실패

1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 1

-   [별 추가]   -
- x 좌표를 입력하시오 : 6
- y 좌표를 입력하시오 : 7
- 별의 이름을 입력하시오 : a
ERROR : 추가 실패

1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 1

-   [별 추가]   -
- x 좌표를 입력하시오 : 3
- y 좌표를 입력하시오 : c
Error Message : For input string: "c"
```

<삭제>

```
1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 2

-      [주어진 별 삭제]      -
- 별의 이름을 입력하시오 : a

1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 3

-      [임의의 별 삭제]      -
별 이름 : c
X 좌표 : 3
Y 좌표 : 3
```

<검색>

```
1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 4

-      [개수 출력]          -
1 개의 별이 존재합니다.

1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 5

-      [이름으로 검색]      -
- 별의 이름을 입력하시오 : b
b 별이 존재합니다.

1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 6

-      [좌표로 검색]        -
- x 좌표를 입력하시오 : 2
- y 좌표를 입력하시오 : 2
(2, 2) 좌표에 별이 존재합니다.
```

<종료>

```
1: 입력  2: 주어진 별 삭제  3: 임의의 별 삭제
4: 출력  5: 이름으로 검색  6: 좌표로 검색          9: 종료
원하는 메뉴를 입력하세요 : 9

-      [종료]              -
1 개의 별이 존재합니다.
< 별의 집합을 종료합니다. >
```

b. 결과분석

$a = (1, 1)$, $b = (2, 2)$, $c = (3, 3)$ 으로 생성하였다.

좌표가 같고 이름이 다른 $x = (1, 1)$ 별을 추가하려 했더니 오류가 났고,

이름이 같고 좌표가 다른 $a = (6, 7)$ 별을 추가하려 했더니 오류가 났으므로

중복을 허용하지 않는 집합의 특성이 잘 드러남을 알 수 있다.

y 좌표에서 c 를 잘못 입력하였을 때엔 parseInt() 함수에서 오류가 남을 try ~ catch 문이 잡아서 에러를 띄워준다.

직접 a 별을 삭제하고 임의 삭제를 이용해 c 별이 삭제되므로 b 별이 남는데,
남은 b 별을 이름과 좌표로 검색해봐도 맞는 결과가 나오며 개수도 1 개가 남는다.

따라서 올바른 결과가 나왔다고 할 수 있다.

4. 소스코드

- StarCollection_ArraySet

AppController.java

```
public class AppController {

    // 상수
    private static final int MENU_ADD = 1;
    private static final int MENU_REMOVE_INPUT = 2;
    private static final int MENU_REMOVE_RANDOM = 3;
    private static final int MENU_PRINT = 4;
    private static final int MENU_SEARCH_NAME = 5;
    private static final int MENU_SEARCH_COORDINATE = 6;
    private static final int MENU_EXIT = 9;

    // 변수
    private AppView _appView;
    private ArraySet<Star> _starCollector;

    // 생성자
    public AppController() {
        this._appView = new AppView();
        this._starCollector = null;
    }

    // Star 추가
    private void inputStar() {

        int xCoordinate, yCoordinate;    // 좌표
        String starName;

        this.showMessage(MessageID.Notice_InputStar);

        // 좌표, 이름 입력
        this.showMessage(MessageID.Notice_InputStarXCoordinate);
        xCoordinate = this._appView.inputInt();

        this.showMessage(MessageID.Notice_InputStarYCoordinate);
        yCoordinate = this._appView.inputInt();

        this.showMessage(MessageID.Notice_InputStarName);
        starName = this._appView.inputString();
    }
}
```

```

        // Star를 starCollector에 추가, 실패 시 에러
        if(!this._starCollector.add(new Star(xCoordinate, yCoordinate, starName))) {
            this.showMessage(MessageID.Error_Input);
        }
    }

    // 이름 검색하여 삭제
    private void removeByName() {

        String starName;

        this.showMessage(MessageID.Notice_RemoveStar);

        this.showMessage(MessageID.Notice_InputStarName);
        starName = this._appView.inputString();

        // starName을 이름으로 가지는 Star 하나 삭제, 실패 시 에러
        if(!this._starCollector.remove(new Star(starName))) {
            this.showMessage(MessageID.Error_Remove);
        }
    }

    // 맨 뒤의 Star 삭제
    private void removeAnyStar() {

        this.showMessage(MessageID.Notice_RemoveRandomStar);
        Star removedStar = this._starCollector.removeAny();

        // 제거된 Star 정보 출력, 실패 시 에러
        if(removedStar != null) {
            this._appView.outputStar(removedStar.starName(),
                                     removedStar.xCoordinate(),
                                     removedStar.yCoordinate());
        }
        else {
            this.showMessage(MessageID.Error_Remove);
        }
    }

    // 이름으로 검색하여 존재여부 확인
    private void searchByName() {

        String starName;

        this.showMessage(MessageID.Notice_SearchByName);

        this.showMessage(MessageID.Notice_InputStarName);
        starName = this._appView.inputString();

        // 이름으로 검색하여 존재할 시 출력, 실패 시 에러
        if(!this._starCollector.contains(new Star(starName))) {
            this._appView.outputMessage("원하는 별이 존재하지 않습니다.");
        }
        else {
            this._appView.outputStarExistence(starName);
        }
    }

```


// 좌표로 검색하여 존재여부 확인

```
private void searchByCoordinate() {  
  
    int xCoordinate, yCoordinate;  
  
    this.showMessage(MessageID.Notice_SearchByCoordinate);  
  
    this.showMessage(MessageID.Notice_InputStarXCoordinate);  
    xCoordinate = this._appView.inputInt();  
  
    this.showMessage(MessageID.Notice_InputStarYCoordinate);  
    yCoordinate = this._appView.inputInt();  
  
    // 좌표로 검색하여 존재할 시 출력, 실패 시 에러  
    if(!this._starCollector.contains(new Star(xCoordinate, yCoordinate))) {  
        this._appView.outputMessage("원하는 별이 존재하지 않습니다.");  
    }  
    else {  
        this._appView.outputStarExistence(xCoordinate, yCoordinate);  
    }  
}
```

// 상황에 맞는 메시지 출력

```
private void showMessage(MessageID aMessageID) {  
  
    switch(aMessageID) {  
  
        // Notices  
        case Notice_StartProgram:  
            this._appView.outputMessage("< 별의 집합을 시작합니다. > \n\n");  
            break;  
  
        case Notice_EndProgram:  
            this._appView.outputMessage("< 별의 집합을 종료합니다. > \n\n");  
            break;  
  
        case Notice_Menu:  
            this._appView.outputMessage("\n");  
            this._appView.outputMessage("1: 입력          2: 주어진 별 삭제  3: 임의의 별 삭제 \n");  
            this._appView.outputMessage("4: 출력          5: 이름으로 검색  6: 좌표로 검색          9: 종료  
\n");  
  
            this._appView.outputMessage("원하는 메뉴를 입력하세요 : ");  
            break;  
  
        case Notice_EndMenu:  
            this._appView.outputMessage("\n- [종료] - \n");  
            break;  
  
        case Notice_InputStar:  
            this._appView.outputMessage("\n- [별 추가] - \n");  
            break;  
  
        case Notice_InputStarName:  
            this._appView.outputMessage("- 별의 이름을 입력하시오 : ");  
            break;  
  
        case Notice_InputStarXCoordinate:  
            this._appView.outputMessage("- x 좌표를 입력하시오 : ");  
            break;
```

```

        case Notice_InputStarYCoordinate:
            this._appView.outputMessage("- y 좌표를 입력하시오 : ");
            break;

        case Notice_RemoveStar:
            this._appView.outputMessage("\n- [주어진 별 삭제] - \n");
            break;

        case Notice_RemoveRandomStar:
            this._appView.outputMessage("\n- [임의의 별 삭제] - \n");
            break;

        case Notice_Show:
            this._appView.outputMessage("\n- [개수 출력] - \n");
            break;

        case Notice_SearchByName:
            this._appView.outputMessage("\n- [이름으로 검색] - \n");
            break;

        case Notice_SearchByCoordinate:
            this._appView.outputMessage("\n- [좌표로 검색] - \n");
            break;

        // Errors
        case Error_WrongMenu:
            this._appView.outputMessage("ERROR : 잘못된 메뉴 번호 \n");
            break;

        case Error_Input:
            this._appView.outputMessage("ERROR : 추가 실패 \n");
            break;

        case Error_Remove:
            this._appView.outputMessage("ERROR : 삭제 실패 \n");
            break;
    }
}

// 시작
public void run() {
    this._starCollector = new ArraySet<Star>();

    this.showMessage(MessageID.Notice_StartProgram);
    int command = 0;

    while(command != MENU_EXIT) {
        try {
            this.showMessage(MessageID.Notice_Menu);
            command = this._appView.inputInt();

            switch(command) {

                case MENU_ADD:
                    this.inputStar();
                    break;

                case MENU_REMOVE_INPUT:
                    this.removeByName();
                    break;
            }
        } catch (Exception e) {
            // TODO: Handle exception
        }
    }
}

```

```

        case MENU_REMOVE_RANDOM:
            this.removeAnyStar();
            break;

        case MENU_PRINT:
            this.showMessage(MessageID.Notice_Show);
            this._appView.outputNumberOfStars(this._starCollector.size());
            break;

        case MENU_SEARCH_NAME:
            this.searchByName();
            break;

        case MENU_SEARCH_COORDINATE:
            this.searchByCoordinate();
            break;

    }

    }

    // Exception catch
    catch(Exception ex) {
        System.out.println("Error Message : " + ex.getMessage());
        continue;
    }

}

this.showMessage(MessageID.Notice_EndMenu);
this._appView.outputNumberOfStars(this._starCollector.size());

this.showMessage(MessageID.Notice_EndProgram);
}

}

```

AppView.java

// 입출력

import java.util.Scanner;

public class AppView {

// 인스턴스 변수

private Scanner _scanner;

// 생성자

```

public AppView() {
    this._scanner = new Scanner(System.in);
}

```

// 입력 메소드

// 숫자 하나 입력받아 반환

```

public int inputInt() {
    return Integer.parseInt(this._scanner.nextLine());
}

```

// 문자열 입력받아 반환

```

public String inputString() {
    return this._scanner.nextLine();
}

```

```

// 출력 메소드

// 메시지 출력
public void outputMessage(String aMessage) {
    System.out.print(aMessage);
}

// Star 정보 출력
public void outputStar(String aStarName, int aX, int aY) {
    System.out.println("별 이름 : " + aStarName);
    System.out.println("X 좌표 : " + aX);
    System.out.println("Y 좌표 : " + aY);
}

// Star 존재 여부 출력 (이름 or 좌표 출력)
public void outputStarExistence(String aStarName) {
    System.out.println(aStarName + " 별이 존재합니다.");
}
public void outputStarExistence(int aX, int aY) {
    System.out.printf("(%d, %d) 좌표에 별이 존재합니다. \n", aX, aY);
}

// 저장된 Star 개수 출력
public void outputNumberOfStars(int aStarCollectorSize) {
    System.out.println(aStarCollectorSize + " 개의 별이 존재합니다.");
}
}

```

ArraySet.java

```

// 집합에 대한 정보

public class ArraySet<E> {

    // 상수
    private static final int DEFAULT_CAPACITY = 100;

    // 인스턴스 변수

    // 집합 최대 용량
    private int _capacity;

    // 현재 집합 용량
    private int _size;

    // 집합은 E의 배열로 구성됨
    private E[] _elements;

    // 생성자

```

```

// 디폴트 : 용량 100
public ArraySet() {
    this._capacity = ArraySet.DEFAULT_CAPACITY;
    this._elements = (E[])new Object[this._capacity];
    this._size = 0;
}

// 용량 직접 설정
public ArraySet(int givenCapacity) {
    this._capacity = givenCapacity;
    this._elements = (E[])new Object[this._capacity];
    this._size = 0;
}

// Getter : 현재 집합의 크기 반환
public int size() {
    return this._size;
}

// 집합 비어있는지 여부 반환
public boolean isEmpty() {
    return (this._size == 0);
}

// 집합 다 찼는지 여부 반환
public boolean isFull() {
    return (this._size == _capacity);
}

// 주어진 order에 있는 원소 반환
public E elementAt(int anOrder) {
    if((0 <= anOrder) && (anOrder < this.size())) {
        return this._elements[anOrder];
    }
    else {
        return null;
    }
}

// 주어진 element과 같은 값의 element이 있는지 여부 반환
public boolean contains(E anElement) {

    boolean found = false;

    for(int index = 0; index < this._size; index++) {
        if(anElement.equals(this._elements[index])) {
            found = true;
            break;
        }
    }

    return found;
}

// 집합에 element 추가
public boolean add(E anElement) {

    if(this.isFull()) {

```

```

        return false;
    }

    // 중복 원소 추가 x
    else if(this.contains(anElement)) {
        return false;
    }

    else {
        this._elements[this._size] = anElement;
        this._size++;
        return true;
    }
}

```

// 집합에서 주어진 값의 element 삭제

```
public boolean remove(E anElement) {
```

```
    boolean found = false;
```

```
    if(this.isEmpty()) {
        return found;
    }

```

// 삭제 시

```
    else {
```

// 집합에 element 1개뿐일 때, 그냥 비우기

```
    if(this._size == 1) {
        found = this._elements[0].equals(anElement);
        this._elements[0] = null;
        this._size--;
    }

```

// 집합에 element가 2개 이상일 때

```
    for(int index = 0; index <= this._size - 1 && !found; index++) {
```

```
        found = this._elements[index].equals(anElement);
```

// 같은 것을 찾으면, 그 인덱스부터 배열 요소 번호들을 1씩 앞으로 당기고 마지막 원소는

null로 초기화

```
        if(found == true) {
            for(int j = index; j <= this._size - 2; j++) {
                this._elements[j] = this._elements[j + 1];
            }
            this._elements[this._size - 1] = null;
            this._size--;
        }
    }

```

```
    return found;
```

```
    }
}

```

// 맨 뒤의 element 하나 삭제하고 삭제한 element 반환

```
public E removeAny() {
```

```
    E removedElement = this._elements[this._size - 1];
    this._elements[this._size - 1] = null;
    this._size--;
    return removedElement;
}

```

// 집합 초기화

```
public void clear() {
```

```
    this._size = 0;
```

```
}  
  
}
```

MessageID.java

```
public enum MessageID {  
  
    // Message IDs for Notices :  
  
    Notice_StartProgram,  
    Notice_EndProgram,  
  
    Notice_Menu,  
    Notice_EndMenu,  
  
    Notice_InputStar,  
    Notice_InputStarName,  
    Notice_InputStarXCoordinate,  
    Notice_InputStarYCoordinate,  
  
    Notice_RemoveStar,  
    Notice_RemoveRandomStar,  
  
    Notice_Show,  
  
    Notice_SearchByName,  
    Notice_SearchByCoordinate,  
  
    // Message IDs for Errors :  
    Error_WrongMenu,  
    Error_Input,  
    Error_Remove  
  
}
```

Star.java

```
// 별에 대한 정보  
  
public class Star {  
  
    // 인스턴스 변수  
  
    // 금액  
    private int _xCoordinate;    // X 좌표  
    private int _yCoordinate;    // Y 좌표  
    private String _starName;    // 별의 이름  
  
    // 생성  
  
    // 디폴트  
    public Star() { }  
  
    // 좌표만 받아 생성  
    public Star(int givenX, int givenY) {  
        this._xCoordinate = givenX;  
        this._yCoordinate = givenY;  
    }  
  
    // 이름만 받아 생성
```

```

public Star(String givenStarName) {
    this._starName = givenStarName;
}

// 좌표, 이름 받아 생성
public Star(int givenX, int givenY, String givenStarName) {
    this._xCoordinate = givenX;
    this._yCoordinate = givenY;
    this._starName = givenStarName;
}

// 공개 함수

// Getter

// 별의 x 좌표 반환
public int xCoordinate() {
    return this._xCoordinate;
}

// 별의 y 좌표 반환
public int yCoordinate() {
    return this._yCoordinate;
}

// 별의 이름 반환
public String starName() {
    return this._starName;
}

// Setter

// x 좌표 설정
public void setXCoordinate(int newX) {
    this._xCoordinate = newX;
}

// y 좌표 설정
public void setYCoordinate(int newY) {
    this._yCoordinate = newY;
}

// 별의 이름 설정
public void setStarName(String newStarName) {
    this._starName = newStarName;
}

// Star 객체끼리의 비교
@Override
public boolean equals(Object aStar) {

    // 인자로 받은 값이 Star 형이 아닐 경우
    if(aStar.getClass() != Star.class) {
        return false;
    }

    // Star형을 받았을 경우, 동일한지 비교
    else {

```



```

        // 좌표 같으면 동일
        if(this._xCoordinate == ((Star)aStar)._xCoordinate &&
            this._yCoordinate == ((Star)aStar)._yCoordinate) {
            return true;
        }

        // 이름 같으면 동일
        if(((Star)aStar)._starName != null &&
            ((Star)aStar)._starName.equals(this._starName)) {
            return true;
        }

        // 둘 다 아니면 다름
        else {
            return false;
        }
    }
}
}
}

```

DS1_05_201702081_최재범.java

```

// main

public class DS1_05_201702081_최재범 {

    public static void main(String[] args) {
        AppController appController = new AppController();
        appController.run();
    }
}

```

- StarCollection_LinkedSet

AppController.java

```

public class AppController {

    // 상수
    private static final int MENU_ADD = 1;
    private static final int MENU_REMOVE_INPUT = 2;
    private static final int MENU_REMOVE_RANDOM = 3;
    private static final int MENU_PRINT = 4;
    private static final int MENU_SEARCH_NAME = 5;
    private static final int MENU_SEARCH_COORDINATE = 6;
    private static final int MENU_EXIT = 9;

    // 변수
    private AppView _appView;
    private LinkedSet<Star> _starCollector;

    // 생성자
    public AppController() {
        this._appView = new AppView();
    }
}

```

```

        this._starCollector = null;
    }

    // Star 추가
    private void inputStar() {

        int xCoordinate, yCoordinate;        // 좌표
        String starName;

        this.showMessage(MessageID.Notice_InputStar);

        // 좌표, 이름 입력
        this.showMessage(MessageID.Notice_InputStarXCoordinate);
        xCoordinate = this._appView.inputInt();

        this.showMessage(MessageID.Notice_InputStarYCoordinate);
        yCoordinate = this._appView.inputInt();

        this.showMessage(MessageID.Notice_InputStarName);
        starName = this._appView.inputString();

        // Star를 starCollector에 추가, 실패 시 에러
        if(!this._starCollector.add(new Star(xCoordinate, yCoordinate, starName))) {
            this.showMessage(MessageID.Error_Input);
        }
    }

    // 이름 검색하여 삭제
    private void removeByName() {

        String starName;

        this.showMessage(MessageID.Notice_RemoveStar);

        this.showMessage(MessageID.Notice_InputStarName);
        starName = this._appView.inputString();

        // starName을 이름으로 가지는 Star 하나 삭제, 실패 시 에러
        if(!this._starCollector.remove(new Star(starName))) {
            this.showMessage(MessageID.Error_Remove);
        }
    }

    // 맨 뒤의 Star 삭제
    private void removeAnyStar() {

        this.showMessage(MessageID.Notice_RemoveRandomStar);
        Star removedStar = this._starCollector.removeAny();

        // 제거된 Star 정보 출력, 실패 시 에러
        if(removedStar != null) {
            this._appView.outputStar(removedStar.starName(),
                                     removedStar.xCoordinate(),
                                     removedStar.yCoordinate());
        }
        else {

```

```

        this.showMessage(MessageID.Error_Remove);
    }

}

// 이름으로 검색하여 존재여부 확인
private void searchByName() {

    String starName;

    this.showMessage(MessageID.Notice_SearchByName);

    this.showMessage(MessageID.Notice_InputStarName);
    starName = this._appView.inputString();

    // 이름으로 검색하여 존재할 시 출력, 실패 시 에러
    if(!this._starCollector.doesContain(new Star(starName))) {
        this._appView.outputMessage("원하는 별이 존재하지 않습니다.");
    }
    else {
        this._appView.outputStarExistence(starName);
    }
}

// 좌표로 검색하여 존재여부 확인
private void searchByCoordinate() {

    int xCoordinate, yCoordinate;

    this.showMessage(MessageID.Notice_SearchByCoordinate);

    this.showMessage(MessageID.Notice_InputStarXCoordinate);
    xCoordinate = this._appView.inputInt();

    this.showMessage(MessageID.Notice_InputStarYCoordinate);
    yCoordinate = this._appView.inputInt();

    // 좌표로 검색하여 존재할 시 출력, 실패 시 에러
    if(!this._starCollector.doesContain(new Star(xCoordinate, yCoordinate))) {
        this._appView.outputMessage("원하는 별이 존재하지 않습니다.");
    }
    else {
        this._appView.outputStarExistence(xCoordinate, yCoordinate);
    }
}

// 상황에 맞는 메시지 출력
private void showMessage(MessageID aMessageID) {

    switch(aMessageID) {

        // Notices
        case Notice_StartProgram:
            this._appView.outputMessage("< 별의 집합을 시작합니다. > \n\n");
            break;

        case Notice_EndProgram:
            this._appView.outputMessage("< 별의 집합을 종료합니다. > \n\n");
            break;
    }
}

```

```

\n");

case Notice_Menu:
    this._appView.outputMessage("\n");
    this._appView.outputMessage("1: 입력          2: 주어진 별 삭제  3: 임의의 별 삭제 \n");
    this._appView.outputMessage("4: 출력          5: 이름으로 검색  6: 좌표로 검색          9: 종료\n");

    this._appView.outputMessage("원하는 메뉴를 입력하세요 : ");
    break;

case Notice_EndMenu:
    this._appView.outputMessage("\n- [종료] - \n");
    break;

case Notice_InputStar:
    this._appView.outputMessage("\n- [별 추가] - \n");
    break;

case Notice_InputStarName:
    this._appView.outputMessage("- 별의 이름을 입력하시오 : ");
    break;

case Notice_InputStarXCoordinate:
    this._appView.outputMessage("- x 좌표를 입력하시오 : ");
    break;

case Notice_InputStarYCoordinate:
    this._appView.outputMessage("- y 좌표를 입력하시오 : ");
    break;

case Notice_RemoveStar:
    this._appView.outputMessage("\n- [주어진 별 삭제] - \n");
    break;

case Notice_RemoveRandomStar:
    this._appView.outputMessage("\n- [임의의 별 삭제] - \n");
    break;

case Notice_Show:
    this._appView.outputMessage("\n- [개수 출력] - \n");
    break;

case Notice_SearchByName:
    this._appView.outputMessage("\n- [이름으로 검색] - \n");
    break;

case Notice_SearchByCoordinate:
    this._appView.outputMessage("\n- [좌표로 검색] - \n");
    break;

// Errors
case Error_WrongMenu:
    this._appView.outputMessage("ERROR : 잘못된 메뉴 번호 \n");
    break;

case Error_Input:
    this._appView.outputMessage("ERROR : 추가 실패 \n");
    break;

case Error_Remove:
    this._appView.outputMessage("ERROR : 삭제 실패 \n");
    break;
}

```

```

    }

    // 시작
    public void run() {
        this._starCollector = new LinkedSet<Star>();

        this.showMessage(MessageID.Notice_StartProgram);
        int command = 0;

        while(command != MENU_EXIT) {
            try {
                this.showMessage(MessageID.Notice_Menu);
                command = this._appView.inputInt();

                switch(command) {

                    case MENU_ADD:
                        this.inputStar();
                        break;

                    case MENU_REMOVE_INPUT:
                        this.removeByName();
                        break;

                    case MENU_REMOVE_RANDOM:
                        this.removeAnyStar();
                        break;

                    case MENU_PRINT:
                        this.showMessage(MessageID.Notice_Show);
                        this._appView.outputNumberOfStars(this._starCollector.size());
                        break;

                    case MENU_SEARCH_NAME:
                        this.searchByName();
                        break;

                    case MENU_SEARCH_COORDINATE:
                        this.searchByCoordinate();
                        break;

                }
            }

            // Exception catch
            catch(Exception ex) {
                System.out.println("Error Message : " + ex.getMessage());
                continue;
            }
        }

        this.showMessage(MessageID.Notice_EndMenu);
        this._appView.outputNumberOfStars(this._starCollector.size());

        this.showMessage(MessageID.Notice_EndProgram);
    }
}

```

AppView.java

// 입출력

import java.util.Scanner;

```

public class AppView {

    // 인스턴스 변수
    private Scanner _scanner;

    // 생성자
    public AppView() {
        this._scanner = new Scanner(System.in);
    }

    // 입력 메소드

    // 숫자 하나 입력받아 반환
    public int inputInt() {
        return Integer.parseInt(this._scanner.nextLine());
    }

    // 문자열 입력받아 반환
    public String inputString() {
        return this._scanner.nextLine();
    }

    // 출력 메소드

    // 메시지 출력
    public void outputMessage(String aMessage) {
        System.out.print(aMessage);
    }

    // Star 정보 출력
    public void outputStar(String aStarName, int aX, int aY) {
        System.out.println("별 이름 : " + aStarName);
        System.out.println("X 좌표 : " + aX);
        System.out.println("Y 좌표 : " + aY);
    }

    // Star 존재 여부 출력 (이름 or 좌표 출력)
    public void outputStarExistence(String aStarName) {
        System.out.println(aStarName + " 별이 존재합니다.");
    }
    public void outputStarExistence(int aX, int aY) {
        System.out.printf("(%d, %d) 좌표에 별이 존재합니다. \n", aX, aY);
    }

    // 저장된 Star 개수 출력
    public void outputNumberOfStars(int aStarCollectorSize) {
        System.out.println(aStarCollectorSize + " 개의 별이 존재합니다.");
    }

}

```

// 집합에 대한 정보

```
public class LinkedSet<Element> {

    private int _size;                // 현재 집이 가지고 있는 원소의 개수
    private Node<Element> _head;      // 맨 앞 노드

    // 생성자
    public LinkedSet() {
        this._size = 0;
        this._head = null;
    }

    // 현재 크기 반환 : getter
    public int size() {
        return this._size;
    }

    // 집합 비어있는지 여부 반환
    public boolean isEmpty() {
        return (this._size == 0 && this._head == null);
    }

    // 집합 다 찼는지 여부 반환 : LinkedSet이라 꼭 차지 않음
    public boolean isFull() {
        return false;
    }

    // 주어진 order에 있는 원소 반환
    public Element elementAt(int givenOrder) {

        Node<Element> currentNode = this._head;

        // order만큼 뒤에 있는 원소의 정보를 currentNode에 저장
        for(int index = 0; index < givenOrder; index++) {
            currentNode = currentNode.next();
        }

        return currentNode.element();
    }

    // 주어진 원소와 같은 값의 원소가 있는지 여부 반환
    public boolean contains(Element givenElement) {

        boolean found = false;
        Node<Element> currentNode = this._head;          // 탐색을 위한 임시 노드, head부터 시작

        // 아직 찾지 못했고, 현재 노드가 존재할 때
        while(!found && (currentNode != null)) {
            if(currentNode.element().equals(givenElement))
                found = true;
            currentNode = currentNode.next();
        }

        return found;
    }
}
```

// 주어진 원소와 같은 값의 원소가 몇 개 있는지 반환

```
public int frequencyOf(Element givenElement) {  
  
    int count = 0;  
    Node<Element> currentNode = this._head;           // 탐색을 위한 임시 노드, head부터 시작  
  
    // 현재 노드가 존재하고, 원소 또한 존재할 때  
    while(currentNode != null && currentNode.element() != null) {  
        if(currentNode.element().equals(givenElement))  
            count++;  
        currentNode = currentNode.next();  
    }  
  
    return count;  
}
```

// 집합에 원소 추가

```
public boolean add(Element givenElement) {  
  
    // 중복 x  
    if(this.contains(givenElement)) {  
        return false;  
    }  
  
    Node<Element> nodeToAdd = new Node<Element>(givenElement, this._head);  
  
    this._head = nodeToAdd;  
    this._size++;  
  
    return true;  
}
```

// 집합에서 주어진 값의 원소 하나 삭제 후 양옆의 노드 연결

```
public boolean remove(Element givenElement) {  
  
    // 비어 있을 때  
    if(this.isEmpty())  
        return false;  
  
    // 노드가 존재  
    else {  
        boolean found = false;  
        Node<Element> previousNode = null;  
        Node<Element> currentNode = this._head;           // 탐색을 위한 임시 노드, head부터 시작  
  
        // 1. 탐색  
  
        // 아직 찾지 못했고, 현재 노드가 존재할 때  
        while(!found && (currentNode != null)) {  
  
            if(currentNode.element().equals(givenElement)) {  
                found = true;  
                break;           // 찾으면 이동 중지  
            }  
  
            previousNode = currentNode;  
            currentNode = currentNode.next();  
        }  
    }  
}
```



```

        // 2. 삭제

        // 찾지 못했을 때
        if(found == false) {
            return false;
        }

        // 찾은 위치가 처음일 때 삭제
        else if(currentNode == this._head)
            this._head = this._head.next();

        // 찾은 위치가 끝일 때 삭제
        else if(currentNode.next() == null)
            previousNode.setNext(null);

        // 찾은 위치가 중간일 때 삭제
        else {
            previousNode.setNext(currentNode.next());
        }

        this._size--;

        return true;
    }
}

// 맨 앞 원소 삭제 후, 삭제한 원소 반환
public Element removeAny() {
    if(this.isEmpty()) {
        return null;
    }
    else {
        Element removedElement = this._head.element();
        this._head = this._head.next();
        this._size--;
        return removedElement;
    }
}

// 집합 초기화
public void clear() {
    this._size = 0;
    this._head = null;
}
}

```

MessageID.java

```

public enum MessageID {

    // Message IDs for Notices :

    Notice_StartProgram,
    Notice_EndProgram,

    Notice_Menu,
    Notice_EndMenu,

    Notice_InputStar,

```

```
Notice_InputStarName,  
Notice_InputStarXCoordinate,  
Notice_InputStarYCoordinate,  
  
Notice_RemoveStar,  
Notice_RemoveRandomStar,  
  
Notice_Show,  
  
Notice_SearchByName,  
Notice_SearchByCoordinate,  
  
// Message IDs for Errors :  
Error_WrongMenu,  
Error_Input,  
Error_Remove
```

```
}
```

Node.java

```
public class Node<Element> {  
  
    private Element _element;           // 현재 노드의 원소  
    private Node<Element> _next;        // 다음 노드  
  
    // 생성자  
  
    // 디폴트  
    public Node() {  
        this._element = null;  
        this._next = null;  
    }  
  
    // 원소만 제공  
    public Node(Element givenElement) {  
        this._element = givenElement;  
        this._next = null;  
    }  
  
    // 원소와 다음 노드 제공  
    public Node(Element givenElement, Node<Element> givenNext) {  
        this._element = givenElement;  
        this._next = givenNext;  
    }  
  
    // Getter  
    public Element element() {  
        return this._element;  
    }  
    public Node<Element> next() {  
        return this._next;  
    }  
  
    // Setter  
    public void setElement(Element givenElement) {  
        this._element = givenElement;  
    }  
    public void setNext(Node<Element> givenNext) {  
        this._next = givenNext;  
    }  
}
```

```
}
```

Star.java

// 별에 대한 정보

public class Star {

 // 인스턴스 변수

 // 금액

private int _xCoordinate; // x 좌표

private int _yCoordinate; // y 좌표

private String _starName; // 별의 이름

 // 생성

 // 디폴트

public Star() { }

 // 좌표만 받아 생성

public Star(**int** givenX, **int** givenY) {
 this._xCoordinate = givenX;
 this._yCoordinate = givenY;
 }

 // 이름만 받아 생성

public Star(**String** givenStarName) {
 this._starName = givenStarName;
 }

 // 좌표, 이름 받아 생성

public Star(**int** givenX, **int** givenY, **String** givenStarName) {
 this._xCoordinate = givenX;
 this._yCoordinate = givenY;
 this._starName = givenStarName;
 }

 // 공개 함수

 // Getter

 // 별의 x 좌표 반환

public int xCoordinate() {
 return this._xCoordinate;
 }

 // 별의 y 좌표 반환

public int yCoordinate() {
 return this._yCoordinate;
 }

 // 별의 이름 반환

public String starName() {
 return this._starName;
 }

```

// Setter

// x 좌표 설정
public void setXCoordinate(int newX) {
    this._xCoordinate = newX;
}

// y 좌표 설정
public void setYCoordinate(int newY) {
    this._yCoordinate = newY;
}

// 별의 이름 설정
public void setStarName(String newStarName) {
    this._starName = newStarName;
}

// Star 객체끼리의 비교
@Override
public boolean equals(Object aStar) {

    // 인자로 받은 값이 Star 형이 아닐 경우
    if(aStar.getClass() != Star.class) {
        return false;
    }

    // Star형을 받았을 경우, 동일한지 비교
    else {

        // 좌표 같으면 동일
        if(this._xCoordinate == ((Star)aStar)._xCoordinate &&
            this._yCoordinate == ((Star)aStar)._yCoordinate) {
            return true;
        }

        // 이름 같으면 동일
        if(((Star)aStar)._starName != null &&
            ((Star)aStar)._starName.equals(this._starName)) {
            return true;
        }

        // 둘 다 아니면 다름
        else {
            return false;
        }
    }
}
}

```

DS1_05_201702081_최재범.java

```

// main

public class DS1_05_201702081_최재범 {

    public static void main(String[] args) {
        AppController appController = new AppController();
        appController.run();
    }
}

```

}